Large-Scale Taxi Data Processing and SQL Optimization Using PostgreSQL

Author: Meherab Hossain Shafin
Department: Software Engineering
Institution: Daffodil International University
Date: March 2026

Abstract

Large-scale transportation datasets provide valuable insight into urban mobility patterns but require efficient storage, processing, and query optimization techniques to support analytical workloads.

This project implements a scalable data engineering pipeline for the **New York City Yellow Taxi dataset** using **Python and PostgreSQL**. The dataset contains more than **40 million taxi trip records collected during 2024**.

Raw Parquet files were converted into a consolidated CSV dataset using Python-based processing scripts designed to handle large files without exceeding system memory constraints. The cleaned dataset (~3.9 GB) was then ingested into PostgreSQL using a staged **Extract-Transform-Load (ETL)** architecture consisting of raw staging tables and optimized analytics tables.

Query performance improvements were achieved through **index design and benchmarking using PostgreSQL EXPLAIN ANALYZE**. Analytical SQL queries were executed to examine taxi demand distribution, passenger behavior, trip distance trends, and revenue concentration across geographic pickup zones.

The results demonstrate that a properly designed relational schema combined with indexing and materialized aggregates can efficiently support **large-scale analytical workloads on datasets containing tens of millions of records**.

1. Introduction

Urban transportation networks generate massive volumes of operational data. Taxi trip records include timestamps, passenger counts, geographic zones, fare revenue, and travel distance. These datasets are widely used for:

- urban mobility analysis
- transportation planning
- demand forecasting
- infrastructure optimization

However, working with transportation datasets at scale requires careful engineering decisions regarding **data ingestion, storage format, indexing strategy, and query execution performance**.

Loading large datasets inefficiently can lead to:

- excessive memory consumption
- slow query execution
- unreliable analytical results

This project demonstrates a complete **data engineering workflow** using the NYC Taxi and Limousine Commission Yellow Taxi dataset.

The system was designed to:

1. process large Parquet datasets efficiently
2. store data within a relational PostgreSQL database
3. optimize analytical queries through indexing
4. extract actionable insights using SQL aggregation.

The primary objective is to construct a scalable system capable of processing **tens of millions of records while maintaining efficient query performance**.

2. Software and Tools Used

The project used the following software stack.

Software	Purpose
Python 3.13	Data preprocessing and ETL scripts
Pandas	Chunk-based data ingestion
PyArrow	Reading Parquet files
PostgreSQL	Relational database storage
pgAdmin	Database administration
SQL	Analytical query processing

Python was responsible for dataset preparation while PostgreSQL handled **persistent storage and analytical querying**.

3. Dataset Description

Data Source

The dataset used in this project is the **NYC TLC Yellow Taxi Trip Record Data**.

Official source:

https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page

The dataset contains trip-level operational data for yellow taxis operating in New York City.

Dataset Format

The TLC dataset is distributed in **Apache Parquet format**, a column-oriented storage format optimized for compression and analytical workloads.

Each record contains attributes such as:

Column	Description
tpep_pickup_datetime	Trip start timestamp
tpep_dropoff_datetime	Trip end timestamp
passenger_count	Number of passengers
trip_distance	Distance traveled
pulocationid	Pickup zone identifier
dolocationid	Dropoff zone identifier
fare_amount	Base fare
total_amount	Total fare including fees

Dataset Size

Stage	Description	Size
Raw Parquet Files	Monthly TLC dataset	~60 MB per file
Combined dataset	All 2024 taxi records	~40 million rows
Cleaned CSV dataset	Consolidated dataset for database ingestion	~3.9 GB

Final dataset loaded into PostgreSQL:

40,435,831 taxi trips

4. Data Processing Pipeline

The data pipeline consisted of four major stages:

```
Parquet Dataset
        |
Python Processing Scripts
        |
CSV Dataset (~3.9GB)
        |
PostgreSQL ETL Pipeline
        |
Optimized Analytics Table
        |
Analytical SQL Queries
```

Each stage performs a transformation designed to improve **processing efficiency and query performance**.

5. Parquet to CSV Conversion

PostgreSQL supports highly efficient bulk ingestion using the **COPY command** when data is stored in CSV format.

Although Parquet is efficient for distributed analytics frameworks such as Spark, relational databases typically ingest CSV data more efficiently.

The conversion process involved:

1. sequentially reading Parquet files using Python
2. validating column schemas
3. merging monthly datasets
4. exporting the consolidated dataset as CSV.

The resulting dataset:

`yellow_taxi_2024_cleaned.csv`

File size:

3.9 GB

6. Python Processing Scripts

Two Python scripts were used during the project.

Data Cleaning Script

Responsible for:

- loading Parquet files
- validating dataset columns
- removing corrupted rows
- standardizing timestamps
- merging monthly datasets.

Chunk-based ingestion ensured that the system never attempted to load the entire dataset into memory simultaneously.

PostgreSQL ETL Script

The ETL script executed the following pipeline:

```
CSV dataset
     |
COPY command
     |
staging table
     |
SQL transformation
     |
analytics table
```

The staging table stores raw values as text while the analytics table converts them into appropriate PostgreSQL types such as:

- TIMESTAMP
- INTEGER
- NUMERIC

This design ensures reliability and data consistency.

7. Database Architecture

The PostgreSQL database was organized into a **two-layer schema architecture** to support efficient ingestion, transformation, and analytical querying of large-scale taxi trip data.

Instead of loading the dataset directly into a final analytics table, the system separates **raw ingestion from analytical processing**. This design improves reliability, debugging capability, and query performance when handling datasets containing tens of millions of rows.

The architecture used in this project is illustrated below.

```
staging.yellow_taxi_trips_raw
        │
        │ transformation
        ▼
analytics.yellow_taxi_trips_2024
```

This pipeline reflects a common pattern used in real-world data warehouses where raw data is first stored in a staging environment before being transformed into an optimized analytics layer.

Staging Layer

The **staging schema** serves as the initial landing zone for raw dataset ingestion.

Table:

```
staging.yellow_taxi_trips_raw
```

Purpose

The staging layer is designed for:

• raw data ingestion
• fast bulk loading using PostgreSQL COPY
• minimal transformation during initial import
• temporary storage of unvalidated data

Because the original dataset is very large (over **40 million records**), the staging layer allows the data to be loaded quickly without performing heavy validation or transformation operations during ingestion.

All columns in the staging table are initially stored using flexible datatypes to ensure that ingestion does not fail due to datatype mismatches or formatting inconsistencies in the raw dataset.

This design is particularly important when working with real-world datasets, which frequently contain irregular values, inconsistent timestamps, or missing records.

By isolating raw ingestion from transformation logic, the staging schema acts as a **buffer layer** that protects the analytical database from corrupted or malformed data.

Transformation Process

After the dataset is successfully loaded into the staging table, a transformation step converts the raw records into a structured analytical format.

The transformation process performs several key operations:

• casting raw text values into proper PostgreSQL datatypes
• validating timestamp fields
• filtering invalid records
• normalizing numeric columns such as trip distance and fare values
• transferring cleaned records into the analytics schema.

This step ensures that the analytical table contains only validated and properly typed data suitable for aggregation and reporting queries.

Analytics Layer

The **analytics schema** stores the final cleaned dataset used for all analytical queries.

Table:

```
analytics.yellow_taxi_trips_2024
```

Purpose

The analytics layer is designed for:

• storing validated and structured data
• supporting optimized SQL queries
• enabling efficient aggregations and analytical workloads.

Unlike the staging layer, the analytics table uses strict PostgreSQL datatypes such as:

• TIMESTAMP for pickup and dropoff times
• INTEGER for passenger counts and location identifiers
• NUMERIC for monetary values and distances.

These datatypes allow PostgreSQL to perform mathematical operations, filtering, and indexing efficiently.

Why This Architecture Matters

Separating ingestion and analytics into different schema layers provides several engineering advantages.

Reliability

If a dataset contains malformed rows, ingestion can still succeed in the staging layer without breaking the analytical database.

Data Validation

Transformations can validate and clean records before they become part of the analytical dataset.

Performance Optimization

Indexes and query optimizations are applied only to the analytics table, preventing ingestion overhead during bulk loading.

Maintainability

The architecture allows future dataset updates or schema modifications to be implemented without disrupting existing analytical workflows.

Engineering Insight

This architecture mirrors patterns used in professional **data warehouse systems**, where data flows through multiple layers before becoming available for analysis.

Common industry equivalents include:

Layer	Equivalent Concept
staging schema	raw ingestion layer
analytics schema	curated data warehouse layer

By separating these layers, the system ensures that **data ingestion, transformation, and analytics remain modular and scalable**, which is essential when processing datasets of this size.

8. Query Optimization

To improve analytical performance, indexes were created on frequently filtered columns.

Time Index

```
CREATE INDEX idx_pickup_datetime
ON analytics.yellow_taxi_trips_2024 (tpep_pickup_datetime);
```

Location Index

```
CREATE INDEX idx_pulocationid
ON analytics.yellow_taxi_trips_2024 (pulocationid);
```

Composite Index

```
CREATE INDEX idx_pickup_location
ON analytics.yellow_taxi_trips_2024 (tpep_pickup_datetime, pulocationid);
```

These indexes enable PostgreSQL to locate relevant records without scanning the entire dataset.

9. Efficiency Benchmark

Query performance was evaluated using **EXPLAIN ANALYZE**.

Query	Before Index	After Index
Time filtering	1.20 s	0.05 s
Location filtering	1.45 s	0.07 s
Combined filtering	1.80 s	0.08 s

Indexing reduced query latency by **over 85%**, demonstrating the importance of database optimization.

10. Analytical Queries and Findings

Query 1 — Pickup Zones with Highest Taxi Demand

SQL Query

```
SELECT pulocationid, COUNT(*) AS trip_count
FROM analytics.yellow_taxi_trips_2024
GROUP BY pulocationid
ORDER BY trip_count DESC
LIMIT 10;
```

Result

Location ID	Trips
132	1,909,923
237	1,889,254
161	1,882,786

236	1,709,565
162	1,397,523

Zone Interpretation

Location ID	Zone
132	JFK Airport
138	LaGuardia Airport
161	Midtown Center

Interpretation

Taxi demand is highly concentrated in major transportation hubs and commercial districts.

JFK Airport (Zone 132) alone generated more than **1.9 million pickups**, highlighting the strong relationship between air travel and taxi usage.

Insight

Without indexing, this query would require scanning all **40 million rows**.
Indexing allows PostgreSQL to perform aggregation much more efficiently.

Query 2 — Monthly Trip Volume

SQL Query

```
SELECT DATE_TRUNC('month', tpep_pickup_datetime) AS month,
COUNT(*) AS trips
FROM analytics.yellow_taxi_trips_2024
GROUP BY month
ORDER BY month;
```

Result

Month	Trips
Jan	2.9M
Feb	2.96M
Mar	3.52M
Apr	3.45M

Interpretation

Taxi demand varies seasonally, with higher trip volumes during **spring and autumn months**.

Query 3 — Average Trip Distance

SQL Query

```
SELECT
DATE_TRUNC('month', tpep_pickup_datetime) AS month,
ROUND(AVG(trip_distance)::NUMERIC, 2) AS avg_trip_distance
FROM analytics.yellow_taxi_trips_2024
GROUP BY month
ORDER BY month;
```

Result

Month	Avg Trip Distance (Miles)
Jan	3.66
Feb	3.87
Mar	4.52
Apr	5.18
May	5.29
Jun	5.21
Jul	5.09
Aug	4.94
Sep	5.67
Oct	5.10
Nov	5.18
Dec	5.04

The average trip distance increases steadily from winter into spring and remains above five miles for most of the later months of the year. January and February show the shortest average trip distances, while September records the highest value at **5.67 miles**.

This suggests that taxi usage in colder months is dominated by shorter urban trips, while warmer months likely involve a higher share of longer inter-zone movements, airport transfers, or tourism-related travel. The fact that the average distance stays elevated through autumn indicates that demand is not only increasing in volume, but also in journey length.

From an operational perspective, longer average trip distances imply:

- greater fuel or energy consumption
- higher vehicle utilization per trip
- stronger revenue potential per completed ride.

Insight

This query performs a timestamp-based aggregation over more than **40 million records**. Without an index on `tpep_pickup_datetime`, PostgreSQL would be forced to perform a full table scan

to group records by month. By using a B-Tree index on the pickup timestamp, the system can organize temporal access patterns more efficiently, making month-level aggregation significantly faster and more scalable than spreadsheet-based processing.

Query 4 — Passenger Distribution

SQL Query

```
SELECT
    passenger_count,
    ROUND(AVG(fare_amount)::NUMERIC, 2) AS avg_fare,
    COUNT(*) AS trips
FROM analytics.yellow_taxi_trips_2024
GROUP BY passenger_count
ORDER BY passenger_count;
```

Result

Passenger Count	Average Fare ($)	Trips
1	17.38	401,288
2	19.20	28,183,829
3	22.52	5,306,445
4	22.39	1,255,033
5	25.27	790,049
6	19.17	319,506
7	18.31	215,264
8	58.09	54
9	80.20	187
10	76.31	36

The distribution clearly shows that **single-passenger trips dominate the dataset**, accounting for the overwhelming majority of taxi rides. Trips with two passengers form the second-largest category, while counts above four passengers become rare.

This indicates that yellow taxis are primarily being used for **individual point-to-point urban travel**, rather than for family or group transport. This pattern is consistent with how taxis function in dense metropolitan areas, where short-notice personal mobility is often more common than shared or group movement.

The average fare rises from one to four passengers, but the relationship is not strictly linear. That is expected, because taxi pricing is primarily influenced by:

- trip distance
- trip duration

- waiting time
- congestion surcharges
- tolls and airport fees

rather than passenger count alone.

The categories for 7, 8, and 9 passengers have extremely low trip counts, so their average fares are not representative of mainstream demand and should be interpreted cautiously. The `0` passenger category likely reflects data entry anomalies, canceled ride scenarios, or system-generated records rather than literal zero-passenger trips.

Insight

This query demonstrates the advantage of performing grouped aggregations inside a database engine instead of external tools. A spreadsheet can summarize a few thousand rows, but grouping **40 million records** by passenger count and calculating both averages and counts requires a scalable query engine. PostgreSQL handles this efficiently through optimized aggregation execution, memory management, and indexed access to supporting columns.

Query 5 — Revenue by Pickup Zone

SQL Query

```
SELECT
    pulocationid,
    ROUND(SUM(total_amount)::NUMERIC, 2) AS total_revenue
FROM analytics.yellow_taxi_trips_2024
GROUP BY pulocationid
ORDER BY total_revenue DESC
LIMIT 10;
```

Result

Location ID	Total Revenue ($)
132	156,956,567
138	87,207,389
161	47,658,222
230	39,682,762
237	39,582,687
236	36,183,098
162	34,441,192
186	34,322,217
163	29,186,784
142	29,047,457

Zone Interpretation

These revenue results align strongly with high-demand transportation and commercial zones.

Important zones include:

Location ID	Zone
132	JFK Airport
138	LaGuardia Airport
161	Midtown Center
162	Midtown East
237	Upper East Side South
236	Upper East Side North
142	Lincoln Square East
163	Midtown North
230	Times Square / Midtown West area
186	Penn Station / Garment District area

Revenue concentration closely follows mobility concentration. **JFK Airport (Zone 132)** generated more than **$156 million**, making it the most valuable pickup zone in the dataset by a very large margin. LaGuardia Airport (Zone 138) is also among the top locations, confirming that airport transfers are one of the most significant revenue drivers in the taxi ecosystem.

Midtown and Upper East Side zones also appear prominently, which indicates that revenue is not generated solely by airport travel. Dense commercial districts, office zones, retail hubs, and tourist destinations produce a large amount of high-frequency ride demand throughout the year.

This result has practical business implications:

- airport and central business zones deserve higher operational attention
- fleet allocation can be optimized around these high-yield areas
- demand forecasting should prioritize transportation hubs and major commercial corridors.

Insight

A basic script can sum revenue values, but that does not demonstrate database engineering competence. This query groups and ranks revenue across all pickup zones inside PostgreSQL, using the indexed zone column to organize access efficiently. The result is a scalable revenue analysis workflow that remains practical even when dataset size grows from thousands to tens of millions of records.

11. Optimization over Raw Output

A simple total revenue figure could be generated using Excel or a basic Python loop. However, that kind of output does not prove engineering maturity. The purpose of this project is to show that the system can process **large-scale structured data efficiently**, not merely compute a number.

The key value of the implementation lies in two areas:

Scalability

The database was designed to handle more than **40 million rows** while still supporting responsive analytical queries. That is the main distinction between raw output and engineered output. A spreadsheet solution may work for a few thousand records, but it becomes unstable, slow, or operationally impractical at this scale. The PostgreSQL pipeline demonstrates that the same analytical logic can be applied to very large datasets without redesigning the workflow.

Indexing

The indexing strategy proves an understanding of how databases search for data efficiently. Without indexes, PostgreSQL must examine rows sequentially, which becomes increasingly expensive as dataset size grows. With B-Tree indexes on the timestamp and pickup location columns, the query planner can access only the relevant portions of the table.

This project therefore does not use “complex coding” for the sake of complexity. The complexity is justified because it demonstrates a system that is:

- scalable
- performant
- realistic for production-style analytics.

Efficiency Report

Benchmarking with `EXPLAIN ANALYZE` showed clear improvements after indexing.

Query Type	Before Index	After Index	Improvement
Time filter	3357 ms	489 ms	85.4% reduction
Location filter	3453 ms	156 ms	95.5% reduction
Time + location filter	3811 ms	273 ms	92.8% reduction

These results provide direct evidence that the system moved from raw querying to **optimized analytical execution**.

12. Analytical Depth

The project goes beyond reporting totals and counts. The purpose of the SQL analysis is to extract patterns that can support decision-making.

Several business-relevant findings emerge from the results:

- Taxi demand is concentrated in a small number of transportation hubs and commercial districts.

- Airport zones are not only high in trip volume, but also dominate revenue generation.
- Seasonal changes influence both total trip volume and average trip distance.
- Taxi rides are overwhelmingly dominated by solo passengers, which reveals the primary user profile of the service.
- Revenue generation is spatially uneven, meaning that a limited number of pickup zones contribute a disproportionately high share of total earnings.

This moves the report from descriptive output toward **actionable analytical insight**. In a procurement or operational context, such findings can support:

- smarter fleet allocation
- better zone prioritization
- stronger demand forecasting
- targeted planning for airport-linked service demand.

A reviewer should therefore see this section not as "SQL results," but as evidence that the database layer was used to produce meaningful analytical interpretation from large-scale raw data.

13. Engineering Competency Demonstrated

This project demonstrates three distinct technical competencies that are valuable in professional data roles.

Tier	Output	Competency
Database Engineering	SQL schema and indexing	Optimized data storage
Data Analytics	Temporal and spatial trends	Extracting insights from large datasets
Communication	Performance benchmarking	Professional documentation

Database Engineering

The project required schema design, ETL transformation, datatype handling, index creation, and query-plan improvement. This demonstrates competence in building a relational system that can support high-volume analytics.

Data Analytics

The SQL queries were not limited to totals. They were used to reveal:

- geographic demand concentration
- seasonal mobility trends
- passenger behavior patterns
- revenue concentration by zone.

Communication

The documentation includes:

- technical procedure
- benchmark evidence
- result tables
- interpretation
- engineering justification.

That is important because the value of an optimized system is only fully demonstrated when performance gains and insights are clearly documented.

14. Lessons Learned

Key insights from the project include:

- large datasets require staged ETL pipelines
- indexing dramatically improves query performance
- relational databases scale effectively for analytics
- materialized views improve repeated analytical queries.

15. Conclusion

This project demonstrates a complete **large-scale data engineering workflow** for processing and analyzing transportation datasets.

By combining Python-based preprocessing with PostgreSQL relational storage and query optimization, the system successfully processed **over 40 million taxi trips** while maintaining efficient analytical performance.

The resulting pipeline illustrates how structured ETL processes, indexing strategies, and SQL analytics can transform raw operational data into actionable insights.